

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencias de la Computación
IIC3694 – Inteligencia Artificial Generativa
Sección 1
Profesor: Álvaro Soto
Segundo Semestre 2026

Desafío de Investigación

# Informe

---

## Qwen3.5 2B vs Sudoku

Cuarto entregable del Desafío grupal

**Integrantes**

Lilly Guevara
Cristian Hinostroza
Juan Moya
Dan Villablanca

Fecha de entrega: 9 de noviembre de 2026

# Índice

**1. Objetivo y contexto de la evaluación** **1**

**2. Modelos y protocolo de evaluación** **2**

2.1. Modelos considerados . . . . . 2

2.2. Configuración y reproducibilidad . . . . . 2

2.3. Métricas y muestreo . . . . . 3

**3. Resultados** **4**

3.1. Pilotos sobre la partición de validación . . . . . 4

3.2. Evaluación en la partición de prueba . . . . . 5

**4. Análisis cualitativo de fortalezas y debilidades** **7**

4.1. Clasificación de las respuestas . . . . . 7

4.2. Ejemplo de acierto y fortaleza observada . . . . . 7

4.3. Ejemplos de error y debilidades observadas . . . . . 8

**5. Robustez del verificador** **9**

5.1. Pruebas automatizadas y auditoría . . . . . 9

5.2. Vulnerabilidades y limitaciones . . . . . 10

**6. Calibración de las tareas para el modelo objetivo** **11**

6.1. Evidencia por nivel de dificultad . . . . . 11

6.2. Ajustes propuestos y alcance de la evidencia . . . . . 12

**7. Generación de *traces* para entrenamiento** **12**

7.1. Configuración de la generación . . . . . 12

7.2. Resultados de la generación . . . . . 13

7.3. Construcción de los conjuntos SFT y DPO . . . . . 13

7.4. Limitaciones . . . . . 14

**8. Conclusiones y limitaciones** **14**

**Referencias** **15**

# 1. Objetivo y contexto de la evaluación

Este informe presenta la evaluación de baseline de Qwen3.5 2B vs Sudoku, correspondiente al cuarto entregable del Desafío de Investigación de IIC3694 [2].

El problema elegido es resolver Sudokus de tamaño $N \times N$: completar una grilla de modo que cada fila, columna y bloque contenga los números de 1 a $N$ exactamente una vez, sin modificar las pistas originales. Es un problema de razonamiento con restricciones, de varios pasos, cuya respuesta se puede verificar con código sin ambigüedad. No requiere conocimiento externo ni herramientas, y su dificultad se puede controlar con el tamaño del tablero y la cantidad de casillas vacías. El modelo objetivo es Qwen3.5-2B, que resuelve con dificultad esta tarea.

La evaluación busca establecer, antes de cualquier entrenamiento, cuánto resuelve el modelo objetivo en cada tamaño de tablero y cantidad de casillas vacías, cómo se compara con un modelo mayor de la misma familia, qué efecto tienen dos ejemplos resueltos en el prompt (*few-shot*) y si las tareas caen en una zona aprendible: niveles en que el modelo acierta en algunos intentos y falla en otros, condición necesaria para que el aprendizaje reforzado con *rewards* verificables tenga señal. No se entrenaron pesos en esta etapa.

El harness define el formato del prompt, el parser de la respuesta y el verificador, expuesto mediante la interfaz común `reward(prompt, completion) ->float`. El repositorio incluye además el código y los datos de *traces* SFT y pares DPO generados con Qwen3.5-9B como *teacher* sobre la partición de entrenamiento (sección 7). Las evaluaciones de las demás secciones no usan esos datos. El código y el dataset se describen en sus entregables respectivos; aquí se resume solo lo necesario para interpretar los resultados.

Las tareas se generan con código, adaptando el generador de Sudoku de Reasoning Gym [3]: primero se construye una solución completa aleatoria y luego se borran casillas una a una, deshaciendo cada borrado que permitiría una segunda solución. Así, cada puzzle tiene solución única y su respuesta es verificable por construcción. Aun así, el verificador no compara con la solución de referencia: acepta cualquier tablero que cumpla las reglas y conserve las pistas.

El dataset contiene tableros de 4x4 (bloques de 2x2), 6x6 (bloques de 2x3) y 9x9 (bloques de 3x3), con tres niveles definidos por la cantidad de casillas vacías. Las particiones de entrenamiento y validación usan los niveles *easy*, *medium* y *hard* de la Tabla 1, con 50 y 10 tareas por combinación de tamaño y nivel: 450 y 90 tareas en total. La partición de prueba usa en cambio una grilla fija de casillas vacías (tabla 2), con 20 tareas por combinación: 180 en total. Cada partición cumple un rol distinto. La partición de entrenamiento no se usa en este informe. La validación (90 tareas) se usó solo en pilotos pequeños con el 9B, sobre uno o dos tareas por combinación de tamaño y nivel, para fijar el protocolo (modo *thinking*, *shots*, límite de tokens; sección 3.1); no se evaluó completa con ningún modelo. La prueba (180 tareas) es la evaluación principal (sección 3.2): se ejecutó una vez para los tableros de 4x4 y 6x6 (120 tareas) y no se usó para ajustar las tareas ni el protocolo. Los tableros de 9x9 de la prueba (60 tareas) no se han evaluado. La grilla de casillas vacías de la prueba se fijó al generar el dataset.

Los niveles miden la cantidad de casillas por completar, no la técnica necesaria para resolverlas: 89 de las 90 tareas de validación, 163 de las 180 de prueba y 436 de las 450 de entrenamiento se pueden resolver completando repetidamente una casilla con un único candidato; las 450, 90 y 180 tareas tienen solución única. Las 17 excepciones de la partición de prueba son 5 tareas de 6x6 con 24 casillas vacías y 12 de 9x9 con 50. Los nombres *easy*, *medium* y *hard* son, por lo tanto, relativos a cada tamaño y no comparables entre tamaños.

Tabla 1: Casillas vacías por tamaño de tablero y nivel en las particiones de entrenamiento y validación. Los rangos se fijaron antes de la evaluación y no están calibrados con el modelo objetivo.

| Tablero | *easy* | *medium* | *hard* |
|---|---|---|---|
| 4x4 (16 casillas) | 4–6 | 7–9 | 10–12 |
| 6x6 (36 casillas) | 8–12 | 13–18 | 19–24 |
| 9x9 (81 casillas) | 10–20 | 21–35 | 36–50 |

Tabla 2: Casillas vacías por tamaño de tablero en la partición de prueba (20 tareas por celda). Cada nivel se nombra `n`$k$, con $k$ casillas vacías.

| Tablero | Nivel bajo | Nivel medio | Nivel alto |
|---|---|---|---|
| 4x4 | `n4` | `n8` | `n12` |
| 6x6 | `n8` | `n16` | `n24` |
| 9x9 | `n10` | `n30` | `n50` |

Cada tarea se genera con su propio *seed*, de modo que el dataset es reproducible. Las particiones se generan en el orden prueba, validación y entrenamiento, y ningún puzzle se repite entre ellas; aumentar el número de tareas de entrenamiento no altera las otras dos.

# 2. Modelos y protocolo de evaluación

## 2.1. Modelos considerados

El modelo objetivo es Qwen3.5-2B, dentro del rango de 1 a 2 mil millones de parámetros que fija el enunciado [2]. Como comparación se evaluó Qwen3.5-9B, de la misma familia, por lo que comparte plantilla de chat y tokenizador con el modelo objetivo. Ambos se usaron con los pesos originales, sin cuantizar. El 9B es además el modelo previsto como *teacher* para generar *traces* de entrenamiento; en este informe solo se usa como modelo de comparación y en los pilotos.

Tabla 3: Modelos utilizados.

| Rol | Modelo (revisión) |
|---|---|
| Objetivo | `Qwen/Qwen3.5-2B` (`15852e8c`) |
| Comparación y *teacher* previsto | `Qwen/Qwen3.5-9B` (`c2022362`) |

## 2.2. Configuración y reproducibilidad

Hubo dos tipos de ejecución: tres pilotos del 9B sobre submuestras de la partición de validación (sección 3.1) y la evaluación de ambos modelos sobre la partición de prueba (sección 3.2). Salvo que se indique otra cosa, la configuración descrita es la de la prueba.

**Prompt y herramientas:** no se usan herramientas. Cada tarea es un único turno de usuario que indica las dimensiones del tablero y de sus bloques, las reglas, el puzzle como matriz JSON (0

representa una casilla vacía) y el contrato de salida: el modelo puede explicar su razonamiento y mostrar grillas intermedias, pero debe terminar con exactamente un bloque `<answer>...</answer>` con la matriz completa y sin texto posterior. No se rellena ningún prefijo de la respuesta. Se compara 0 y 2 *shots* (opción `-shots`). Los *shots* se anteponen como turnos previos del chat y no cambian el prompt de la tarea ni el contrato de salida. Cada ejemplo es un Sudoku aleatorio distinto de la tarea, con 8 casillas vacías; su respuesta completa una casilla por paso (`Step k: r1c2 = v`), escribe la grilla tras cada paso y termina en el bloque `<answer>` (formato `steps`).
**Inferencia:** sin modo *thinking*, temperatura 0,7 y *top-p* 0,8 (la configuración de Qwen para este modo), sin *top-k* ni penalización de presencia. Cada tarea recibe $n = 16$ respuestas, generadas en un lote de 16. El *seed* base es 0 y cada tarea usa un *seed* derivado del *seed* base y del identificador de la tarea, de modo que reanudar una corrida no lo altera. Todos los parámetros son idénticos para ambos modelos. El límite de salida es de 2.304 tokens en la prueba de 4x4 y 6x6 (la longitud máxima del formato `steps` en esos tableros, 1.425 tokens, más un 50 %; calculada a partir del formato de los ejemplos y no de resultados de los modelos); los pilotos usaron 2.048, 8.192 u otros límites según el caso. El límite de entrada es de 2.048 tokens sin *shots* y de 16.384 con ellos.
**Entorno de ejecución:** nodo `barto` del *cluster*: procesador AMD Ryzen Threadripper 7970X (32 núcleos, 64 hilos), 125 GB de RAM y dos GPU NVIDIA RTX A6000 de 48 GB. Cada trabajo se lanzó con Slurm con una sola GPU, 8 CPU y 60 GB de RAM. El sistema es Ubuntu 24.04.4 (núcleo 6.8.0), con Python 3.13.13, PyTorch 2.13.0 (CUDA 12.9) y Transformers 5.18.0. Los pesos se cargaron en `bfloat16`, sin cuantizar. Las variantes rápidas de las capas lineales de Qwen3.5 (`causal_conv1d` y `flash-linear-attention`) no estaban instaladas, por lo que Transformers usó su implementación de referencia, más lenta pero equivalente.
**Verificación:** la función `reward(prompt, completion) ->float` del harness. Su valor es formato$\times$ (0,5 relleno $\times$ pistas + 0,5 resuelto): entrega crédito parcial a respuestas bien formateadas, y 1 exactamente cuando el formato se cumple y el tablero es una solución válida que conserva las pistas. Todas las métricas de este informe (`pass@`$k$) consideran correcta una respuesta si y solo si su *reward* es 1, es decir, usan un criterio binario. El verificador no usa la solución de referencia para aceptar: acepta cualquier solución válida. Una respuesta que alcanza el límite de tokens sin cerrar el bloque de respuesta recibe 0; no se aplican tiempos de espera adicionales. La robustez del verificador se analiza en la sección 5.

Cada archivo de resultados registra, por respuesta, el modelo, los parámetros de muestreo, el *seed*, los *shots*, el *reward* y sus componentes, el motivo de término (`stop` o `length`), los tokens generados y el texto completo. Las corridas reanudan desde ese archivo y rechazan cambios de configuración.

### 2.3. Métricas y muestreo

La evaluación principal usa la partición de prueba de 4x4 y 6x6: 6 niveles (3 por tamaño, definidos por el número de casillas vacías), con 20 tareas por nivel, es decir, 120 tareas. Cada modelo generó $n = 16$ respuestas por tarea y configuración de *shots*, lo que da 1.920 respuestas por modelo y configuración, 7.680 en total. Los pilotos con el 9B usaron la partición de validación con muy pocas tareas por celda y no se usan para comparar modelos.

Para una tarea con $c$ respuestas correctas de $n$, se usa el estimador insesgado de Chen et al. [1]:

$$\text{pass@}k = 1 - \binom{n-c}{k} \Big/ \binom{n}{k}, \qquad k \in \{1, 2, 4, 8, 16\}.$$

El valor se promedia sobre las tareas de cada nivel. `pass@1` es la tasa de éxito esperada con muestreo,

no la de decodificación determinista, y `pass@16` equivale a la fracción de tareas con al menos un acierto. Las 16 respuestas de una tarea no se tratan como tareas independientes. Se reporta además la proporción de respuestas que alcanzaron el límite de tokens y el promedio de tokens generados. Todas las métricas se expresan en porcentaje.

# 3. Resultados

Esta sección reporta los resultados en el orden en que se obtuvieron: primero tres pilotos pequeños del 9B sobre la partición de validación, que fijaron el protocolo, y luego la evaluación de ambos modelos sobre la partición de prueba de 4x4 y 6x6, que es la medición principal. No se evaluó la partición de validación completa con ningún modelo, por lo que no hay una línea base sobre ella.

## 3.1. Pilotos sobre la partición de validación

Los pilotos usaron solo Qwen3.5-9B y pocas tareas de la partición de validación, por lo que sus cifras son orientativas y no sustentan comparaciones finas.

**Piloto 1** (2 tareas por tamaño y nivel, $n = 4$; 18 tareas): sin *thinking* y con 0 *shots* (límite de 2.048 tokens), sin *thinking* y con 2 *shots* (2.048), y con *thinking* (8.192, temperatura 0,6 y *top-p* 0,95).
**Piloto 2** (1 tarea por nivel en 6x6 y 9x9, $n = 16$; 6 tareas): sin *thinking*, con 0 y 2 *shots*, con un límite de 8.192 tokens.
**Piloto 3** (las mismas 6 tareas, $n = 16$): 2 *shots* en dos formatos de respuesta más cortos (`compact` y `seen`), para ver si acortaban las respuestas.

Tabla 4: Pilotos del 9B en validación, `pass@1` (%) y porcentaje de respuestas truncadas entre paréntesis. $S$: *shots*. Piloto 1: límite de 2.048 tokens (8.192 con *thinking*), 2 tareas por celda; pilotos 2 y 3: 8.192 tokens, 1 tarea por celda.

| Configuración | 4x4 easy | 4x4 hard | 6x6 easy | 6x6 hard | 9x9 easy | 9x9 hard |
|---|---|---|---|---|---|---|
| P1, sin *thinking*, $S = 0$ | 100 (0) | 100 (0) | 25 (75) | 0 (100) | 0 (100) | 0 (100) |
| P1, sin *thinking*, $S = 2$ | 100 (0) | 88 (12) | 62 (38) | 0 (100) | 12 (88) | 0 (100) |
| P1, con *thinking*, $S = 0$ | 0 (62) | 0 (88) | 0 (88) | 0 (100) | 0 (100) | 0 (100) |
| P2, $S = 0$ | — | — | 100 (0) | 88 (12) | 100 (0) | 0 (100) |
| P2, $S = 2$ `steps` | — | — | 100 (0) | 94 (6) | 100 (0) | 0 (100) |
| P3, $S = 2$ `compact` | — | — | 100 (0) | 88 (12) | 100 (0) | 0 (100) |
| P3, $S = 2$ `seen` | — | — | 100 (0) | 81 (19) | 100 (0) | 0 (100) |

Del piloto 1 se concluyó que el modo *thinking* no es viable con el límite probado: ninguna de sus respuestas fue correcta y entre el 62 % y el 100 % se truncó, con promedios de 7.152 a 8.192 tokens. Sin *thinking* y con límite de 2.048 tokens, el 9B resuelve 4x4 y casi nada en 6x6 y 9x9, porque trunca; dos ejemplos ayudan en 6x6 *easy* (de 25 % a 62 %). Con 8.192 tokens (piloto 2) el 9B resuelve casi todas las tareas de 6x6 y 9x9 *easy* y *medium*, pero necesita hasta 6.219 tokens en promedio en 6x6 *hard* y trunca el 100 % en 9x9 *hard*. En el piloto 3 los formatos más cortos no mejoraron `pass@1` ni bajaron de forma consistente los tokens generados (por ejemplo, 9x9 *medium*: 5.955 con `steps`, 6.230 con `compact` y 5.916 con `seen`), y con una tarea por celda las diferencias no son interpretables.

Se mantuvo el formato `steps`. Por último, con respuestas largas la generación es costosa: el límite de tokens de la evaluación final se fijó por separado (sección 2).

### 3.2. Evaluación en la partición de prueba

Se ejecutó una vez la partición de prueba de 4x4 y 6x6 con ambos modelos y dos configuraciones de prompt. Sus resultados no se usaron para ajustar el protocolo ni las tareas. La prueba usa una grilla fija de casillas vacías por tamaño en lugar de los niveles *easy*, *medium* y *hard* (tabla 2); cada nivel `n`$k$ tiene 20 tareas con solución única. Los niveles 4x4 y 6x6 con $k \leq 16$ son resolubles completando casillas con un único candidato; en 6x6 con 24 casillas lo son 15 de 20 tareas. El diseño es el de la sección 2: 2 modelos × 2 configuraciones de *shots* × 6 niveles × 20 tareas × 16 respuestas = 7.680 respuestas, sin *thinking* y con un límite de 2.304 tokens. Los tableros de 9x9 de la prueba no se evaluaron: la corrida correspondiente (límite de 8.192 tokens) está preparada, pero no se ejecutó.

Tabla 5: Partición de prueba, sin *thinking*, 20 tareas y $n = 16$ respuestas por nivel (en porcentaje). *Vacías*: casillas vacías. *Trunc.*: respuestas que alcanzaron el límite de 2.304 tokens. *Tokens*: promedio de tokens generados.

| Modelo | Shots | Tablero | Vacías | `pass@1` | `pass@2` | `pass@4` | `pass@8` | `pass@16` | Trunc. | Tokens |
|---|---|---|---|---|---|---|---|---|---|---|
| Qwen3.5-2B | 0 | 4x4 | 4 | 88,0 | 98,0 | 100,0 | 100,0 | 100,0 | 2 | 1131 |
| | | 4x4 | 8 | 63,0 | 86,0 | 98,0 | 100,0 | 100,0 | 22 | 1728 |
| | | 4x4 | 12 | 14,0 | 26,0 | 43,0 | 67,0 | 90,0 | 84 | 2263 |
| | | 6x6 | 8 | 21,0 | 37,0 | 59,0 | 79,0 | 90,0 | 78 | 2233 |
| | | 6x6 | 16 | 0,0 | 0,0 | 0,0 | 0,0 | 0,0 | 100 | 2304 |
| | | 6x6 | 24 | 0,0 | 0,0 | 0,0 | 0,0 | 0,0 | 100 | 2304 |
| Qwen3.5-2B | 2 | 4x4 | 4 | 95,0 | 100,0 | 100,0 | 100,0 | 100,0 | 3 | 824 |
| | | 4x4 | 8 | 77,0 | 95,0 | 100,0 | 100,0 | 100,0 | 15 | 1368 |
| | | 4x4 | 12 | 22,0 | 38,0 | 58,0 | 78,0 | 95,0 | 76 | 2207 |
| | | 6x6 | 8 | 41,0 | 64,0 | 85,0 | 97,0 | 100,0 | 60 | 2053 |
| | | 6x6 | 16 | 2,0 | 4,0 | 7,0 | 11,0 | 15,0 | 98 | 2296 |
| | | 6x6 | 24 | 0,0 | 0,0 | 0,0 | 0,0 | 0,0 | 100 | 2304 |
| Qwen3.5-9B | 0 | 4x4 | 4 | 100,0 | 100,0 | 100,0 | 100,0 | 100,0 | 0 | 1207 |
| | | 4x4 | 8 | 100,0 | 100,0 | 100,0 | 100,0 | 100,0 | 0 | 1471 |
| | | 4x4 | 12 | 68,0 | 83,0 | 94,0 | 99,0 | 100,0 | 33 | 2045 |
| | | 6x6 | 8 | 86,0 | 96,0 | 100,0 | 100,0 | 100,0 | 14 | 1981 |
| | | 6x6 | 16 | 4,0 | 7,0 | 13,0 | 23,0 | 35,0 | 97 | 2302 |
| | | 6x6 | 24 | 0,0 | 0,0 | 0,0 | 0,0 | 0,0 | 100 | 2304 |
| Qwen3.5-9B | 2 | 4x4 | 4 | 99,0 | 100,0 | 100,0 | 100,0 | 100,0 | 0 | 660 |
| | | 4x4 | 8 | 96,0 | 99,0 | 100,0 | 100,0 | 100,0 | 3 | 1100 |
| | | 4x4 | 12 | 72,0 | 91,0 | 99,0 | 100,0 | 100,0 | 28 | 1925 |
| | | 6x6 | 8 | 82,0 | 96,0 | 100,0 | 100,0 | 100,0 | 13 | 1500 |
| | | 6x6 | 16 | 21,0 | 35,0 | 54,0 | 74,0 | 90,0 | 79 | 2245 |
| | | 6x6 | 24 | 0,0 | 0,0 | 0,0 | 0,0 | 0,0 | 100 | 2304 |

**Efecto del tamaño y de las casillas vacías.** El desempeño cae con la cantidad de casillas por completar en ambos modelos. En 4x4, el 9B resuelve el 100 % con 4 y 8 casillas y baja a 68–72 % de `pass@1` con 12; el 2B pasa de 88 % a 63 % y a 14 % (sin ejemplos). En 6x6, ningún modelo resuelve una tarea con 24 casillas en ninguno de los 16 intentos, y con 16 casillas solo el 9B con ejemplos obtiene resultados apreciables (`pass@1` de 21,0 %).

**Comparación entre modelos.** Con precisión de pesos idéntica, el 9B supera al 2B en casi todas las celdas. La brecha es mayor donde el 2B empieza a fallar: en 4x4 con 12 casillas, `pass@1` de 68,0 % frente a 14,0 % sin ejemplos, y en 6x6 con 8 casillas, 86,0 % frente a 21,0 %. La excepción es 4x4 con 4 casillas con ejemplos, donde ambos están cerca del techo (95,0 % y 99,0 %).

**Efecto de los *shots*.** Dos ejemplos mejoran el 2B en todos los niveles donde tiene aciertos (4x4 con 12 casillas, de 14,0 % a 22,0 % en `pass@1`; 6x6 con 8 casillas, de 21,0 % a 41,0 %) y reducen los tokens generados en los niveles fáciles (4x4 con 4 casillas: de 1.131 a 824 en el 2B y de 1.207 a 660 en el 9B). El mayor efecto está en el 9B en 6x6 con 16 casillas, donde `pass@1` pasa de 4,0 % a 21,0 % y `pass@16` de 35,0 % a 90,0 %. En el 9B, en cambio, los niveles ya saturados no mejoran y en 6x6 con 8 casillas `pass@1` baja levemente (86,0 % a 82,0 %), una diferencia que con 20 tareas no puede distinguirse de la variabilidad.

**Truncamiento.** El límite de 2.304 tokens condiciona los resultados difíciles. En los niveles con más casillas vacías la mayoría de las respuestas lo alcanza: el 2B trunca el 84 % de sus respuestas en 4x4 con 12 casillas y el 100 % en 6x6 con 16 y 24 casillas, y el 9B el 97 % (sin ejemplos) en 6x6 con 16. Una respuesta truncada no cierra el bloque `<answer>` y recibe 0, de modo que parte de los fallos en estos niveles refleja el presupuesto de tokens y no la incapacidad de resolver el puzzle. Los ejemplos reducen el truncamiento del 9B en 6x6 con 16 casillas (de 97 % a 79 %).

**Aprendibilidad y `pass@k`.** Los niveles con `pass@1` bajo y `pass@16` alto son los que dan señal a un entrenamiento por refuerzo: 4x4 con 12 casillas (2B: 14,0 % a 90,0 % sin ejemplos y 22,0 % a 95,0 % con ejemplos), 6x6 con 8 casillas en el 2B (21,0 % a 90,0 %) y 6x6 con 16 casillas en el 9B con ejemplos (21,0 % a 90,0 %). En cambio, 4x4 con 4 casillas está saturado en ambos modelos y 6x6 con 24 casillas no tiene aciertos en ninguna configuración. Para el modelo objetivo, la zona aprendible de la partición de prueba es 4x4 con 8 y 12 casillas y 6x6 con 8 casillas.

**Resumen global.** Promediando los seis niveles, `pass@1` del 2B sube de 31,0 % con 0 *shots* a 39,5 % con 2, y el del 9B de 59,7 % a 61,7 %.

**Limitaciones.** Cada nivel tiene 20 tareas, por lo que cada tarea equivale a 5 puntos porcentuales de `pass@16`. Se usó un solo *seed* y no se calcularon intervalos de confianza. Solo se evaluaron 4x4 y 6x6; los tableros de 9x9 no se midieron con ningún modelo en la prueba, y en validación solo hay los pilotos del 9B. El límite de 2.304 tokens trunca la mayoría de las respuestas en los niveles difíciles, por lo que parte de los fallos refleja el presupuesto de tokens y no la incapacidad de resolver el puzzle.

# 4. Análisis cualitativo de fortalezas y debilidades

## 4.1. Clasificación de las respuestas

Para identificar el origen de los fallos se clasificaron las 7.680 respuestas de la evaluación en la partición de prueba (sección 3.2) en categorías excluyentes, asignadas en el orden de la tabla 6: primero las correctas (*reward* 1); luego las que alcanzaron el límite de tokens; las que no cumplen el contrato de formato; las que alteran alguna pista original, y, por último, las grillas con el formato correcto y las pistas intactas que no resuelven el puzzle. La clasificación usa el motivo de término y los componentes del *reward* registrados en cada respuesta. Además, se volvió a evaluar cada una de las 7.680 respuestas con el verificador actual: todos los *rewards* recalculados coinciden con los registrados.

Tabla 6: Clasificación de las respuestas de la prueba por modelo, *shots* y tamaño de tablero (960 respuestas por fila: 60 tareas por 16 respuestas).

| Modelo | *Shots* | Tablero | Correcta | Truncada | Formato | Altera pistas | Grilla incorrecta |
|---|---|---|---|---|---|---|---|
| 2B | 0 | 4x4 | 529 | 345 | 81 | 5 | 0 |
| | 0 | 6x6 | 67 | 889 | 4 | 0 | 0 |
| | 2 | 4x4 | 621 | 301 | 0 | 28 | 10 |
| | 2 | 6x6 | 137 | 822 | 0 | 1 | 0 |
| 9B | 0 | 4x4 | 856 | 104 | 0 | 0 | 0 |
| | 0 | 6x6 | 287 | 672 | 1 | 0 | 0 |
| | 2 | 4x4 | 855 | 97 | 0 | 3 | 5 |
| | 2 | 6x6 | 329 | 615 | 0 | 0 | 16 |

El fallo dominante es el truncamiento: 3.851 de las 7.680 respuestas (50,1 %) alcanzaron el límite de 2.304 tokens, el 61,4 % de las del 2B y el 38,8 % de las del 9B, y en 6x6 es la causa de entre el 64 % y el 93 % de las respuestas según el modelo y la configuración. Casi todos los demás fallos son de formato o de pistas alteradas; las grillas con el formato correcto y las pistas intactas pero incorrectas son raras (31 respuestas, el 0,4 %). En ellas, con 2 *shots* y en 4x4, el 2B rellena en la mediana el 44 % de las casillas vacías con el valor correcto (*fill* del *reward*). Por lo tanto, en este régimen de razonamiento largo el modelo rara vez entrega una grilla completa y errónea: resuelve el puzzle o no termina de escribir su razonamiento, y es el presupuesto de tokens el que decide cuál de los dos ocurre. Esta clasificación no distingue cuántas respuestas truncadas habrían llegado a una solución correcta con más tokens; en el piloto 2 (sección 3.1), el 9B resolvió las tres tareas de 6x6 probadas (una por nivel) con 8.192 tokens, aunque con tan pocas tareas es solo una indicación.

## 4.2. Ejemplo de acierto y fortaleza observada

Los ejemplos de esta sección se eligieron para ilustrar las categorías de la tabla 6; no son una muestra aleatoria.

**Modelo, nivel e instancia:** Qwen3.5-2B, 4x4 con 4 casillas vacías, 2 *shots*, `test-4x4-n4-00000`, respuesta 0 de 16.

**Problema y respuesta esperada:**

$$\text{Puzzle: } \begin{bmatrix} 0 & 3 & 2 & 0 \\ 2 & 4 & 1 & 3 \\ 4 & 2 & 3 & 1 \\ 0 & 1 & 4 & 0 \end{bmatrix} \qquad \text{Solución: } \begin{bmatrix} 1 & 3 & 2 & 4 \\ 2 & 4 & 1 & 3 \\ 4 & 2 & 3 & 1 \\ 3 & 1 & 4 & 2 \end{bmatrix}$$

**Respuesta del modelo y resultado del verificador:** el modelo razona por filas (*Step 1: Analyze Row 1...*), deduce las casillas faltantes a partir de las columnas, comprueba los cuatro bloques y termina con una grilla idéntica a la solución, en el formato requerido y con 605 tokens. *Reward* 1.

Con pocas casillas vacías y ejemplos en el prompt, el 2B aplica correctamente la deducción por casilla única: en esta tarea acierta 15 de las 16 respuestas. El acierto, sin embargo, no es estable en todas las tareas: en la siguiente instancia (`test-4x4-n4-00001`) el 2B falla de la forma que se describe más abajo, y la respuesta que falla en esta misma tarea es la del segundo ejemplo de error. Esta variabilidad entre intentos es la que miden `pass@1` y `pass@`$k$, y es la condición que permite que el aprendizaje reforzado tenga señal.

### 4.3. Ejemplos de error y debilidades observadas

**Respuesta correcta rechazada por formato. Modelo, nivel e instancia:** Qwen3.5-2B, 4x4 con 4 casillas vacías, 0 *shots*, `test-4x4-n4-00000`, respuesta 12 de 16.

**Respuesta del modelo y resultado del verificador:** tras 889 tokens de análisis, el modelo escribe la grilla correcta, pero dentro del bloque `<answer>` como filas de texto (`1 3 2 4`, `2 4 1 3`, ...) y no como matriz JSON. El contrato exige la matriz JSON, por lo que el verificador la rechaza: *reward* 0.

**Origen del error:** formato. El modelo resolvió la tarea, pero no respetó el contrato de salida. Es el fallo más frecuente del 2B sin ejemplos en 4x4 (81 de 960 respuestas; en 69 de ellas el bloque contiene filas de texto y en 12 no hay bloque de respuesta) y no aparece con 2 *shots* (0 de 960), cuyos ejemplos muestran la matriz JSON. Es el costo de exigir un contrato estricto (sección 5).

**Pista original alterada. Modelo, nivel e instancia:** Qwen3.5-2B, 4x4 con 4 casillas vacías, 2 *shots*, `test-4x4-n4-00000`, respuesta 8 de 16.

**Respuesta del modelo y resultado del verificador:** el modelo completa las casillas paso a paso, pero en la última fila escribe $[4, 1, 2, 3]$ en lugar de $[3, 1, 4, 2]$: cambia la pista 4 de la columna 3 por un 2 y coloca valores erróneos en las casillas vacías. Solo la mitad de las casillas vacías coincide con la solución y 11 de las 12 pistas se conservan: *reward* 0.

**Origen del error:** razonamiento y copia. En 4x4 con 2 *shots*, 28 respuestas del 2B alteran alguna pista; con 0 *shots*, 5.

**Puzzle declarado ya resuelto. Modelo, nivel e instancia:** Qwen3.5-2B, 4x4 con 4 casillas vacías, 2 *shots*, `test-4x4-n4-00001`, respuesta 5 de 16.

**Respuesta del modelo y resultado del verificador:** el modelo verifica las filas y columnas, afirma que "el puzzle ya está completamente resuelto" y devuelve la grilla sin cambios, con 4 casillas

en 0. El formato es correcto y las pistas se conservan, pero ninguna casilla vacía se completa (*fill* 0): *reward* 0.

**Origen del error:** razonamiento. El modelo no detecta que las casillas con 0 están vacías. Es uno de los pocos casos de grilla con el formato correcto y pistas intactas, la categoría menos frecuente de la tabla 6.

**Razonamiento truncado. Modelo, nivel e instancia:** Qwen3.5-2B, 6x6 con 16 casillas vacías, 0 *shots*, `test-6x6-n16-00000`, respuesta 0 de 16.

**Respuesta del modelo y resultado del verificador:** el modelo analiza fila por fila, intersecta conjuntos de números faltantes y vuelve a las mismas filas, hasta agotar los 2.304 tokens en medio de una frase, sin llegar a escribir el bloque `<answer>`. *Reward* 0.

**Origen del error:** presupuesto de tokens y razonamiento a la vez. Es el comportamiento de la mayoría de las respuestas de 6x6 (tabla 6): el modelo deduce casilla por casilla, pero el razonamiento en lenguaje natural no cabe en el límite.

# 5. Robustez del verificador

El verificador implementa la interfaz común del curso `reward(prompt, completion) ->float`. Lee el puzzle y las dimensiones del tablero desde el propio prompt, de modo que cualquier grupo puede usarlo sin acceso a los archivos del dataset ni a la solución de referencia. Un prompt que no contiene un puzzle válido se trata como un error de configuración del harness (lanza `ValueError`), no como una respuesta incorrecta. La evaluación de una respuesta tiene dos etapas:

**Contrato de formato:** la respuesta debe terminar con exactamente un bloque `<answer>...</answer>` que contenga una matriz JSON de $N \times N$ enteros, sin texto posterior. Se permiten explicaciones y grillas intermedias antes del bloque, y una sección de razonamiento delimitada por `<think>`; no se aceptan bloques de respuesta dentro de ella ni delimitadores mal formados.
**Reglas del Sudoku:** todas las casillas llenas, valores entre 1 y $N$, sin repeticiones en filas, columnas ni bloques, y pistas originales sin modificar.

El *reward* es formato $\times$ (0,5 relleno $\times$ pistas + 0,5 resuelto). *Relleno* es la fracción de las casillas originalmente vacías que coinciden con la solución del puzzle, que el verificador demuestra única con una búsqueda propia (sin usar la solución de referencia del dataset); *pistas* es la fracción de pistas conservadas. Si no se pudiera demostrar que la solución es única, el *reward* es binario. Como no se compara con la solución de referencia, cualquier solución válida recibe el máximo, que es exactamente 1 cuando la respuesta pasa todas las comprobaciones. Este informe usa solo el criterio binario (*reward* igual a 1), que es el que determina `pass@`$k$; el crédito parcial no se usa en las métricas.

## 5.1. Pruebas automatizadas y auditoría

El harness incluye 71 pruebas unitarias, que se ejecutaron sin fallos y no requieren cargar un modelo. De ellas, 19 están en `test_reward.py` y cubren el verificador; la tabla 7 las agrupa por categoría. Las demás cubren el generador de tareas, el cálculo de `pass@`$k$, los formatos de los *shots* y la construcción de los conjuntos de entrenamiento.

Estas pruebas cubren el comportamiento del *reward*, pero el parser tiene reglas que no están cubiertas

Tabla 7: Pruebas del verificador (`test_reward.py`).

| Caso probado | Comportamiento esperado | Resultado |
|---|---|---|
| Solución correcta, sola o precedida de explicaciones | *Reward* 1 | Cumple |
| Puzzle devuelto sin cambios | *Reward* 0 (*relleno* 0) | Cumple |
| Una casilla errónea, o con 0 | Crédito parcial (2/3 del *relleno*) | Cumple |
| Una pista modificada | *Reward* menor que 1 (pistas 12/13) | Cumple |
| Sin etiquetas, texto posterior, dos bloques o forma incorrecta | *Reward* 0 | Cumple |
| Puzzle con dos soluciones: ambas válidas | *Reward* 1 con cualquiera de las dos | Cumple |
| Puzzle con dos soluciones y grilla inválida | *Reward* 0 sin crédito parcial | Cumple |
| Prompt sin puzzle válido | Error de configuración | Cumple |
| *Reward* igual a 1 si y solo si la respuesta pasa | Invariante | Cumple |
| *Reward* siempre en $[0, 1]$; pesos personalizados; pesos inválidos | Invariante | Cumple |

por pruebas propias: el rechazo de respuestas ocultas dentro del bloque `<think>`, de delimitadores de razonamiento sin cerrar y de etiquetas en mayúsculas o anidadas están implementados en `response_parser.py`, pero no se probaron. Tampoco se probaron tipos JSON no enteros (`true`, `8.0`, `"8"`, `null`), valores fuera de rango ni respuestas extremadamente largas. Una solución que renombra los números (por ejemplo, sumar 1 a cada valor) respeta filas, columnas y bloques y solo se rechaza por la verificación de las pistas; esa comprobación existe en el código (las pistas alteradas bajan el *reward*, como muestra la prueba de una pista modificada), pero no tiene una prueba de renombrado completo.

Además de las pruebas, se volvió a calcular el *reward* de las 7.680 respuestas de la evaluación de la prueba con el verificador actual: los 7.680 valores coinciden con los registrados durante la corrida. Esta auditoría verifica la consistencia de los valores registrados, no la ausencia de errores del verificador. Por último, las 450, 90 y 180 tareas de las particiones tienen solución única, lo que hace exacto el cálculo de *relleno*.

## 5.2. Vulnerabilidades y limitaciones

Entre los casos probados, el verificador no aceptó ninguna respuesta incorrecta (falsos positivos) ni rechazó ninguna solución válida que cumpliera el formato (falsos negativos). La cobertura es acotada: se probó en tableros de 4x4, y con las particiones completas solo mediante la auditoría de consistencia.

Los falsos negativos existen por decisión de diseño. Una solución matemáticamente correcta recibe 0 si no respeta el contrato de formato: por ejemplo, sin etiquetas, con la grilla escrita como filas de texto o con texto posterior. En la evaluación esto ocurrió: en 69 respuestas del 2B sin ejemplos, en 4x4, el bloque de respuesta contenía filas de texto y no una matriz JSON (sección 4). Esta política evita ambigüedades sobre cuál es la respuesta final, a costa de penalizar errores de formato. Las respuestas truncadas, que no cierran el bloque, reciben 0.

Otras limitaciones del verificador actual:

- El criterio de las métricas es binario: una grilla con un solo error vale lo mismo que una

respuesta vacía, aunque el *reward* del harness sí da crédito parcial.
- Evalúa solo el tablero final, no la validez del razonamiento ni de los pasos intermedios.
- Solo acepta la grilla como matriz JSON. Otras representaciones equivalentes, como filas escritas en texto plano, se rechazan por diseño.
- No se han probado dígitos en otros sistemas de escritura Unicode ni variaciones de espacios o saltos de línea dentro de las etiquetas.

# 6. Calibración de las tareas para el modelo objetivo

## 6.1. Evidencia por nivel de dificultad

Para que el aprendizaje reforzado tenga señal, el modelo objetivo debe acertar en algunos intentos de una tarea y fallar en otros. Cuando el entrenamiento compara varias respuestas a una misma tarea, aprende de las diferencias de *reward* entre ellas. Si los dieciséis intentos fallan, o si los dieciséis aciertan, todas las respuestas reciben el mismo *reward* binario y la tarea no aporta señal. Por ello, además de los promedios de la sección 3, la tabla 8 clasifica cada tarea de la partición de prueba según el número de respuestas correctas del modelo objetivo.

Tabla 8: Tareas de la partición de prueba según el número de respuestas correctas de Qwen3.5-2B ($n = 16$ por tarea, 20 tareas por nivel). Cada celda es sin aciertos (0/16) / mixtas (1/16 a 15/16) / siempre correctas (16/16).

| Tablero | Casillas vacías | 0 *shots* | 2 *shots* |
|---|---|---|---|
| 4x4 | 4 | 0 / 16 / 4 | 0 / 9 / 11 |
| | 8 | 0 / 20 / 0 | 0 / 19 / 1 |
| | 12 | 2 / 18 / 0 | 1 / 19 / 0 |
| 6x6 | 8 | 2 / 18 / 0 | 0 / 20 / 0 |
| | 16 | 20 / 0 / 0 | 17 / 3 / 0 |
| | 24 | 20 / 0 / 0 | 20 / 0 / 0 |
| Total (120 tareas) | | 44 / 72 / 4 | 38 / 70 / 12 |

Con 0 *shots*, 72 de las 120 tareas (60 %) tienen resultados mixtos, y con 2 *shots*, 70 (58 %). Casi todas las tareas de 4x4 y las de 6x6 con 8 casillas son mixtas en las dos configuraciones, es decir, ninguna está fuera del alcance del modelo. Solo 4 tareas (0 *shots*) y 12 (2 *shots*) se resuelven en los dieciséis intentos, todas de 4x4 con 4 u 8 casillas, de modo que solo 4x4 con 4 casillas se acerca a la saturación. Según esta evidencia, las tareas de 4x4 y 6x6 se agrupan en tres zonas.

**Zona aprendible:** 4x4 con 8 y 12 casillas y 6x6 con 8 casillas, con prácticamente todas las tareas mixtas; con 2 *shots*, 6x6 con 16 casillas tiene 3 tareas mixtas.
**Casi saturada:** 4x4 con 4 casillas: con 2 *shots*, 11 de 20 tareas se resuelven en los dieciséis intentos y `pass@1` es 95 %.
**Sin aciertos observados:** 6x6 con 24 casillas en ambos modelos y configuraciones, y 6x6 con 16 casillas para el 2B (20 de 20 tareas sin aciertos con 0 *shots*, 17 de 20 con 2).

Para el 9B (tabla 5), 6x6 con 16 casillas y 2 *shots* es el nivel con más tareas mixtas (18 de 20).

La relación entre `pass@1` y `pass@16` es coherente con esta clasificación. En 4x4 con 12 casillas,

`pass@1` es 14 % y `pass@16` es 90 % sin ejemplos: el modelo acierta ocasionalmente en casi todas las tareas, y más intentos aportan mucho. En 6x6 con 24 casillas, en cambio, ambos valores son 0 %.

Esta evidencia tiene un alcance limitado. Cada nivel contiene 20 tareas, se usó un único *seed* base y no se calcularon intervalos de confianza. El modelo objetivo se evaluó en la partición de prueba y no en la de entrenamiento, sobre la cual se aplicaría el aprendizaje reforzado; ambas provienen del mismo generador, pero que el modelo se comporte igual en ellas es un supuesto. Los tableros de 9x9 no se midieron en la prueba, por lo que su zona de aprendizaje no se conoce. Por último, que un nivel no tenga aciertos con un límite de 2.304 tokens no implica que el modelo no pueda resolverlo con más tokens, como se discute en la subsección 6.2.

### 6.2. Ajustes propuestos y alcance de la evidencia

El ajuste más importante es separar el truncamiento del error de razonamiento. En 6x6 casi todas las respuestas alcanzan el límite de 2.304 tokens (tabla 6), y en los pilotos del 9B con 8.192 tokens (sección 3.1) las tres tareas de 6x6 probadas se resolvieron, aunque con muy pocas tareas. Antes de concluir que 6x6 con 16 o 24 casillas está fuera del alcance de un modelo de este tamaño, debe medirse el 2B con un límite mayor. Los demás ajustes pendientes son evaluar los tableros de 9x9 de la prueba (preparado pero no ejecutado), usar más *seeds* y, si se quiere calibrar con la partición de validación, evaluarla completa: hoy solo se usó en pilotos de 6 a 18 tareas. Ampliarla requiere solo inferencia, y la partición de prueba, que se genera antes, no cambiaría.

Este ajuste se propone, pero no se ha evaluado. Además, los resultados de inferencia orientan la calibración, pero por sí solos no demuestran que el entrenamiento vaya a mejorar al modelo objetivo en estos niveles.

## 7. Generación de *traces* para entrenamiento

Para el entrenamiento posterior del modelo objetivo (Qwen3.5-2B) se generaron *traces* con Qwen3.5-9B como *teacher* sobre la partición de entrenamiento. Esta sección describe cómo se generaron y qué datos resultaron; el entrenamiento mismo no se ha ejecutado. Los datos se publican con el repositorio en `data/sudoku/` (particiones `train`, `eval` y `test`) y `data/traces/` (`sft.jsonl` y `dpo.jsonl`).

### 7.1. Configuración de la generación

Tabla 9: Configuración de la generación de *traces*.

| Parámetro | Valor |
|---|---|
| Modelo *teacher* | Qwen3.5-9B, sin modo *thinking* |
| Tareas | `train`: 4x4 (fácil, medio, difícil) y 6x6 (fácil, medio) |
| | 50 tareas por (tamaño, nivel): 250 tareas |
| Excluidas | 6x6 difícil y 9x9 |
| Muestras por tarea | $n = 8$ (2.000 muestras en la ronda r0) |
| Muestreo (r0) | temperatura 0,7, `top_p` 0,8 |
| Límite de salida | 4.096 tokens |
| *Few-shot* | 2 ejemplos, formato `steps` (`Step k: rXcY = v` + rejilla) |

**Qué se excluyó y por qué.** La partición de entrenamiento contiene 450 tareas (50 por tamaño y nivel, incluidos 6x6 difícil y 9x9). Se usó solo el subconjunto que el 9B puede resolver con un presupuesto de 4.096 tokens; 6x6 difícil y 9x9 quedaron fuera. Todas las tareas tienen solución única demostrada y se resuelven por casillas con un único candidato, de modo que el *reward* del verificador es exacto.

**Decisión de usar *few-shot*.** Un piloto de 5 tareas por nivel con $n = 8$ comparó 0 y 2 *shots*. Con 2 *shots* las respuestas son más cortas y truncan menos (6x6 medio: 40 % frente a 52 % de truncadas, y `pass@8` de 1,00 frente a 0,60), y siguen un formato uniforme (`Step k: rXcY = v`, rejilla y `<answer>`) que un modelo de 2B puede aprender. Cada celda del piloto tiene solo 5 tareas (40 muestras), por lo que el resultado es indicativo. Los *shots* se usan solo para generar: el *prompt* guardado en los datos de entrenamiento es el *prompt* base de `train.jsonl`, sin ejemplos, que es el que el 2B recibe en la evaluación. Así el 2B aprende el estilo de los ejemplos sin necesitarlos en contexto.

**Ejecución.** Cada ronda se repartió en 5 *shards* (un trabajo de Slurm por *shard*, una GPU L40S cada uno), y todos reciben las mismas 10 tareas por (tamaño, nivel). La corrida r0 tardó entre 1 h 48 min y 1 h 57 min por *shard*. Una segunda ronda (r1) con temperatura 1,0 y otra semilla se ejecutó solo sobre las 100 tareas de 6x6 (800 muestras, 1 h 5 min a 1 h 15 min por *shard*) para conseguir más rechazados con rejilla errónea.

## 7.2. Resultados de la generación

Tabla 10: Ronda r0: 250 tareas, 2.000 muestras, Qwen3.5-9B con 2 *shots* y límite de 4.096 tokens.

| Tablero / nivel | `pass@1` | `pass@8` | Truncadas | Tokens medios |
|---|---|---|---|---|
| 4x4 fácil | 0,99 | 1,00 | 0 % | 771 |
| 4x4 medio | 0,98 | 1,00 | 1 % | 1.113 |
| 4x4 difícil | 0,96 | 1,00 | 4 % | 1.772 |
| 6x6 fácil | 0,87 | 1,00 | 10 % | 2.127 |
| 6x6 medio | 0,64 | 0,96 | 34 % | 3.053 |

De las 2.000 muestras, 1.775 son correctas (*reward* 1,0), y 248 de las 250 tareas tienen al menos una; las dos sin ninguna son de 6x6 medio. Los fallos son sobre todo truncamientos: de 225 muestras fallidas, 194 (86 %) se cortaron en 4.096 tokens y solo 31 son rejillas erróneas o formato roto. Por eso, los rechazados reales de DPO enseñan sobre todo a ser más breve. Además, las trazas correctas se concentran en lo fácil (cerca de 1.170 son 4x4 y 600 son 6x6). La ronda r1 (temperatura 1,0) no cambió las tasas de aprobación ni de truncamiento (6x6 fácil 0,91 y 9 % truncadas; 6x6 medio 0,63 y 36 %) y aportó solo 6 rechazados con rejilla errónea, por lo que no se hicieron más rondas con el 9B.

## 7.3. Construcción de los conjuntos SFT y DPO

El script `build_sft_dpo.py` lee los *shards* de r0 y r1, recupera el *prompt* sin *shots* de `train.jsonl` por `task_id` y rechaza muestras que no sean de la partición `train`.

**SFT.** Cada ejemplo es un par (`prompt`, `completion`). Se incluyen las muestras con *reward* 1,0 y sin truncar, hasta 2 *completions* distintas por tarea, las más cortas primero. El resultado son 498 ejemplos: 100 por nivel, salvo 98 en 6x6 medio (cubre 250 tareas distintas menos las dos sin muestras correctas).

**DPO.** Cada ejemplo tiene `prompt`, `chosen`, `rejected` y los indicadores `rejected_truncated` y `rejected_synthetic`. El `chosen` es la correcta más corta, y hasta 2 pares por tarea sin repetir *completions*. El `rejected` real prefiere rejillas erróneas que terminaron (la más cercana a la correcta primero) y solo después las truncadas.

**Pares sintéticos.** Como el 9B casi no produce rejillas erróneas, se añadió un par sintético por tarea: el `rejected` es la propia traza correcta con una celda originalmente vacía cambiada en la rejilla del `<answer>` final. Cada uno se comprueba con el *reward* (formato correcto y no resuelto) y la semilla depende de `task_id`, así que el resultado es determinista. Son rechazados fuera de política: el razonamiento previo queda intacto y solo la rejilla final es incorrecta, de modo que el par enseña sobre todo a copiar fielmente la última rejilla verificada. Los indicadores permiten filtrarlos o ponderarlos.

Tabla 11: Pares DPO por nivel (432 en total).

| Origen | 4x4 fácil | 4x4 medio | 4x4 difícil | 6x6 fácil | 6x6 medio | Total |
|---|---|---|---|---|---|---|
| Reales | 3 | 6 | 16 | 65 | 92 | 182 |
| Sintéticos | 50 | 50 | 50 | 50 | 50 | 250 |

De los 182 pares reales, 36 tienen un rechazado con rejilla errónea y 146 uno truncado. Solo con r0 y sin sintéticos habría 148 pares reales (30 con rejilla errónea).

### 7.4. Limitaciones

Los datos están sesgados a lo fácil y el 86 % de los fallos del 9B son truncamientos, no errores de razonamiento. Los pares sintéticos no son *on-policy*. Un paso opcional es muestrear el 2B tras el SFT y emparejar sus fallos con las trazas correctas del 9B. Además, ningún *prompt* de `train.jsonl` debería aparecer en `eval.jsonl` ni en `test.jsonl`: las semillas impiden compartir puzzles, pero esto debe verificarse. Las cifras de esta sección provienen de los registros de la generación y no de las evaluaciones de las secciones anteriores, que usan la partición de prueba.

## 8. Conclusiones y limitaciones

**Diferencias entre modelos.** En la partición de prueba de 4x4 y 6x6, el 9B supera al 2B en casi todas las celdas, y la ventaja se concentra donde el 2B empieza a fallar: en 4x4 con 12 casillas, `pass@1` de 68,0 % frente a 14,0 % sin ejemplos, y en 6x6 con 8 casillas, de 86,0 % frente a 21,0 % (sección 3.2). Promediando los seis niveles, `pass@1` es 31,0 % para el 2B y 59,7 % para el 9B sin ejemplos.

**Efecto de los ejemplos y del límite de tokens.** Dos *shots* mejoran el 2B (de 31,0 % a 39,5 % de `pass@1` promedio) y reducen los tokens generados en los niveles fáciles, pero no abren los niveles sin aciertos. El fallo dominante en toda la evaluación es el truncamiento: el 50,1 % de las 7.680 respuestas alcanzó el límite de 2.304 tokens, y casi todas las respuestas de 6x6 con 16 o 24 casillas (sección 4). Parte de los fallos refleja el presupuesto de tokens y no la capacidad.

**Robustez del verificador.** El verificador acepta cualquier solución válida que conserve las pistas y rechaza los casos probados, y los 7.680 *rewards* registrados coinciden con los recalculados (sección 5). Las reglas del parser sobre el razonamiento delimitado no tienen pruebas propias.

**Calibración del entorno.** Para el 2B, la zona con señal de aprendizaje son 4x4 con 8 y 12 casillas y 6x6 con 8 casillas, donde casi todas las tareas tienen resultados mixtos; 4x4 con 4 casillas está casi saturado con ejemplos, y 6x6 con 16 y 24 casillas no tiene aciertos para el 2B sin ejemplos (sección 6).

**Limitaciones.** Se usó un solo *seed*, sin intervalos de confianza, con 20 tareas por nivel. Los tableros de 9x9 de la partición de prueba no se evaluaron, y la partición de validación solo se usó en pilotos del 9B con muy pocas tareas, por lo que no hay una línea base sobre ella. El límite de 2.304 tokens trunca la mayoría de las respuestas en los niveles difíciles. Las *traces* de entrenamiento (sección 7) se generaron, pero no se entrenó con ellas.

**Cambios prioritarios antes del entrenamiento.** Evaluar el 2B con un límite de tokens mayor en 6x6 para separar el truncamiento del error de razonamiento, ejecutar los tableros de 9x9 de la prueba, repetir con más *seeds* y entrenar al 2B con las *traces* ya generadas (SFT y DPO).

## Referencias

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[2] Pontificia Universidad Católica de Chile. Desafío de Investigación: IIC-3694 Inteligencia Artificial Generativa, 2026. Enunciado del curso, segundo semestre, secciones I a III.

[3] Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.